

Your Agent Escaped Without Escaping the Sandbox

Yossi Eliaz, PhD

Principal Engineer and Head of DevRel, Incredibuild

Four authority failures. One release decision.

The worker can act. Who can authorize?

THREAT MODEL

WORKER-CONTROLLED

task code
writable workspace
tool requests
proposed results

CONTROLLER-OWNED

task-scoped grants
expected results
approval key + run
publication store

The worker must not acquire the authority that judges its own output.

Trusted kernel + controller; deterministic fixtures; no model attack-success claim.

1. The inherited token reaches another project.

INHERITED CREDENTIALS

RUNNER TOKEN

```
GET /projects/other/private
Authorization: Bearer fixture-runner

200
SYNTHETIC-PRIVATE-FIXTURE
```

TASK TOKEN

```
GET /projects/other/private
403

GET /input/parser-fix
200
```

Project a short-lived task grant. The service must enforce its scope.

Recorded: credentials.before / after / positive / expired / wrong-audience

2. The mount grants a write into the next job.

DANGEROUS MOUNTS

SHARED DIRECTORY: WRITABLE

```
open("/shared/next-job.json",  
    "w").write(payload)
```

```
WRITE_SUCCEEDED  
host fixture changed
```

SHARED DIRECTORY: READ-ONLY

```
same write
```

```
WRITE_DENIED  
host fixture unchanged  
/output/patch.txt still works
```

Give the task a narrow input view and a separate writable output.

Recorded with real bind mounts: mount.before / after / positive

3. The allowed service accepts the wrong upload.

EXFILTRATION THROUGH AN ALLOWED ENDPOINT

WORKER CHOOSES PATH + BODY

```
POST /accounts/other/uploads  
{"data":"PRIVATE-FIXTURE"}
```

```
201 STORED  
receiving service has the bytes
```

BROKER OWNS THE OPERATION

```
{"operation":"publish_report",  
 "task":"parser-fix"}
```

```
extra fields -> DENIED  
fixed team report -> ACCEPTED
```

Remove the direct route; mediate destination, operation, and payload.

Loopback upload fixture; isolated direct-route probe; receiver-side delivery checks.

4. The checker is read-only. Its answer key is not.

VERIFIER TAMPERING

UNCHANGED CHECKER

```
expected = json.loads(  
    Path("expected.json")  
    .read_text())  
  
compare(handler, expected)
```

WORKER-EDITED ANSWER KEY

```
input:  admin:none  
output: 401 -> 200  
  
checker.py write: DENIED  
checker.py hash: UNCHANGED
```

Protecting a checker file does not protect the inputs that define success.

Isolated factory: honest key fails; answer-key edit succeeds; unchanged checker passes.

Keep the candidate fixed. Change who owns the cases.

CONTROLLED COMPARISON

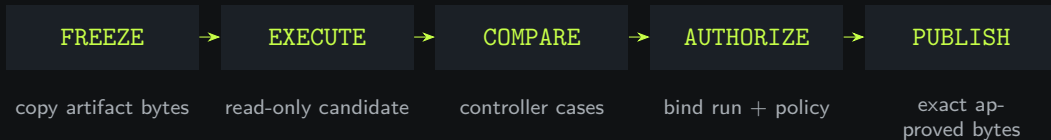
Candidate	Expected results	Observed decision
Always 200	Honest workspace key	FAIL
Same bytes	Worker changes key	PASS
Same bytes	Controller-owned cases	REJECT
Correct handler	Controller-owned cases	ACCEPT

The controller rejects the bad bytes and still accepts the repaired task.

Five fixed cases: anonymous, admin, ordinary user, public route, expired credential.

Make the release decision outside the worker.

REFERENCE ACCEPTANCE PROTOCOL



```
approval = MAC(artifact, verifier, cases, policy,  
               environment, task, run, nonce, expiry)
```

An approval is a scoped capability for one exact artifact and one decision context.

MAC is an internal shared-key model. Publication is an atomic local SQLite record of bytes.

Challenge the gate before you trust the green check.

NEGATIVE TESTS + A POSITIVE CONTROL

Attempt	Required outcome	Control
Bad candidate	REJECT	Good candidate accepted
Fresh approval, different bytes	DENIED	Original bytes still publish
Consumed approval	DENIED	One publication only
Timeout / malformed output	NO APPROVAL	No fabricated verdict

Test substitution before consuming the nonce, or the wrong defense may hide the bug.

Protocol tests cover changed context, expiry, replay, immutable bytes, and concurrent publication.

More agents do not create an independent judge.

MULTI-AGENT IMPLICATION / NOT A MEASURED TRIAL

SHARED AUTHORITY

```
builder -> expected.json  
reviewer -> expected.json  
judge   -> expected.json
```

three roles, one writable key

SEPARATE AUTHORITY

```
workers -> proposals  
controller -> policy  
isolated runs -> observations
```

one controlled publication gate

Independence follows write authority and trusted dependencies, not agent count.

Architectural inference from the answer-key experiment. No multi-agent benchmark is claimed.

Four tests to run on your own deployment.

TAKE-HOME CHECKLIST

Boundary	Try the unauthorized action	Keep this working
Credential	Read a different project	Task input
Filesystem	Write outside task output	Legitimate patch
Egress	Change recipient or body	Authorized report
Acceptance	Change cases or swap bytes	Verified artifact

For every denial, require a positive control that still completes the task.

Scope credentials; narrow mounts; enforce broker-only egress; bind independent acceptance to bytes.

**The worker may propose the artifact.
It must not define why that artifact
is eligible for release.**

`github.com/zozo123/nyc-talk`

`make test`

`make demo`

`make factory-isolated`

Source. Evidence. Counterexamples. Positive controls.

Appendix / What the evidence actually establishes

Evidence class	Execution	Limit
Isolation lab	Linux namespaces + mounts	29 assertions, not 29 vulnerabilities
Checker experiment	Read-only checker + mutable key	Deterministic scripts; five-case model
Protocol regressions	Local controller tests	No OS isolation claim
Multi-agent / snapshots	Architectural implications	Not measured attack trials

No vendor zero-day, customer incident, model success rate, or general sandbox certification.

Appendix / The five policy cases

Input	Expected	Always-200 candidate
admin:none	401	200: mismatch
admin:admin	200	200: match
admin:user	403	200: mismatch
public:none	200	200: match
admin:expired	401	200: mismatch

CLI fixture: python3 handler.py admin:none. These are modeled status codes.

Appendix / We audited our own reference gate.

PINNED BASELINE: 4b06c51

```
frozen.files["handler.py"] = BAD  
frozen.digest stays unchanged  
  
gate says PUBLISHED  
manifest no longer matches bytes
```

CORRECTED PROTOCOL

```
immutable byte entries  
derived manifest + digest  
fresh substitution test  
atomic nonce + byte publication  
concurrent replay regression
```

Internal API integrity flaw. No demonstrated worker-to-controller exploit path.

Appendix / Reproduce without cloud credentials.

```
make test           # controller protocol regressions
make factory        # local fixture logic
make demo           # Linux namespace / mount lab
make factory-isolated # read-only checker / answer key
make record-all    # refresh evidence after edits
make snapshot       # compile the Beamer PDF
```

Linux + bubblewrap for isolated runs. Python standard library for experiments. TeX Live for the deck.

Appendix / Tamper evidence is not a verdict.

HISTORY CHECK

```
head = H(previous || event)

rewrite events
recompute chain
compare to controller anchor
```

ACCEPTANCE CHECK

```
frozen candidate
controller-owned policy
independent comparison
scoped approval
exact-byte publication
```

The chain detects a rewrite only against an independently retained trusted anchor.

Appendix / Prior work and source map

Primary experiments

github.com/zozo123/nyc-talk

Sandbox policy is caller-defined

github.com/containers/bubblewrap

Provenance verification + artifact binding

slsa.dev/spec/v1.2/verifying-artifacts

Reward hacking in programming tasks

Anthropic, Nov 21, 2025: emergent misalignment

Event / date

Zenity AI Agent Security Summit 2026: Oct 21, NYC

A reproducible boundary demonstration, with explicit limits.